

ASSIGNMENT 7

Page Object Model (POM) Restructure

STUDENT NAME:	Roshni Ghosh Tagore
ENROLLMENT NO:	12023002028175
COURSE:	Python for Automation
ACADEMIC YEAR:	2023–2027
DEPARTMENT:	Computer Science and Engineering (Artificial Intelligence & Machine Learning)
CAMPUS:	IEM Newtown

1. PROBLEM STATEMENT

Restructure monolithic Selenium test scripts into a modular Page Object Model (POM) design pattern. The implementation must strictly separate element locator definitions and UI interaction methods into independent page classes, while keeping test logic, assertions, and driver execution flows completely isolated inside a dedicated test script.

2. OBJECTIVE

- Implement the **Page Object Model (POM)** design pattern to enhance code reusability, maintainability, and architectural cleanliness.
- Create encapsulated page classes (LoginPage and DashboardPage) containing locator definitions and action wrapper methods.
- Verify successful user navigation and UI header title validation (Products) on the inventory page using assertions in the execution script without hardcoding locators in the main test logic.

3. TOOLS, SOFTWARE, AND CONCEPTS USED

Technologies & Libraries:

Python 3.x

Selenium WebDriver

Google Chrome

By Class Locators

Core Technical Concepts:

- **Design Pattern:** Page Object Model (POM)
- **Object-Oriented Programming:** Class structure, encapsulation, constructor initialization (`__init__`), object instantiation.
- **Python Dynamic Expressions:** Tuple unpacking (`*self.locator`) for passing locator strategies directly to `find_element`.
- **Target Application:** SauceDemo E-Commerce Platform (<https://www.saucedemo.com/>)

4. SOURCE CODE IMPLEMENTATION

File 1: `pages/dashboard_page.py`

```
from selenium.webdriver.common.by import By

class DashboardPage:

    page_title = (By.CLASS_NAME, "title")

    def __init__(self, driver):
        self.driver = driver

    def get_page_title(self):
        return self.driver.find_element(*self.page_title).text
```

File 2: pages/login_page.py

```
from selenium.webdriver.common.by import By

class LoginPage:

    # Locators
    username = (By.ID, "user-name")
    password = (By.NAME, "password")
    login_button = (By.XPATH, "//input[@id='login-button']")

    def __init__(self, driver):
        self.driver = driver

    # UI methods
    def enter_username(self, username):
        self.driver.find_element(*self.username).send_keys(username)

    def enter_password(self, password):
        self.driver.find_element(*self.password).send_keys(password)

    def click_login(self):
        self.driver.find_element(*self.login_button).click()

    def login(self, username, password):
        self.enter_username(username)
        self.enter_password(password)
        self.click_login()
```

File 3: Assignment7.py (Test Script)

```
from selenium import webdriver
from pages.login_page import LoginPage
from pages.dashboard_page import DashboardPage

driver = webdriver.Chrome()

driver.get("https://www.saucedemo.com/")
driver.maximize_window()

# Create LoginPage object
login_page = LoginPage(driver)

# Login
login_page.login("standard_user", "secret_sauce")

# Create DashboardPage object
dashboard_page = DashboardPage(driver)

# Assertion
assert "/inventory.html" in driver.current_url

print("Login successful")
print("Dashboard title:", dashboard_page.get_page_title())

driver.quit()
```

5. OUTPUT WITH CLEARLY LABELLED SCREENSHOTS

```
pages > dashboard_page.py > DashboardPage > get_page_title
1  from selenium.webdriver.common.by import By
2
3
4  class DashboardPage:
5
6      page_title = (By.CLASS_NAME, "title")
7
8      def __init__(self, driver):
9          self.driver = driver
10
11      def get_page_title(self) (function) find_element: Any
12          return self.driver.find_element(*self.page_title).text
```

Screenshot 1: Dashboard Page Object Class (pages/dashboard_page.py)

Defines locator strategies for dashboard components and encapsulates UI text extraction methods.

```
pages > login_page.py > LoginPage
1  from selenium.webdriver.common.by import By
2
3
4  class LoginPage:
5
6      # Locators
7      username = (By.ID, "user-name")
8      password = (By.NAME, "password")
9      login_button = (By.XPATH, "//input[@id='login-button']")
10
11      def __init__(self, driver):
12          self.driver = driver
13
14      # UI methods
15      def enter_username(self, username):
16          self.driver.find_element(*self.username).send_keys(username)
17
18      def enter_password(self, password):
19          self.driver.find_element(*self.password).send_keys(password)
20
21      def click_login(self):
22          self.driver.find_element(*self.login_button).click()
23
24      def login(self, username, password):
25          self.enter_username(username)
26          self.enter_password(password)
27          self.click_login()
28
29
```

Screenshot 2: Login Page Object Class (pages/login_page.py)

Encapsulates login page elements (ID, NAME, XPATH) and interactive wrapper methods.

```

Assignment7.py > ...
1  from selenium import webdriver
2  from pages.login_page import LoginPage
3  from pages.dashboard_page import DashboardPage
4
5
6  driver = webdriver.Chrome()
7
8  driver.get("https://www.saucedemo.com/")
9  driver.maximize_window()
10
11  # Create LoginPage object
12  login_page = LoginPage(driver)
13
14  # Login
15  login_page.login("standard_user", "secret_sauce")
16
17  # Create DashboardPage object
18  dashboard_page = DashboardPage(driver)
19
20  # Assertion
21  assert "/inventory.html" in driver.current_url
22
23  print("Login successful")
24  print("Dashboard title:", dashboard_page.get_page_title())
25
26  driver.quit()

```

Screenshot 3: Main Test Execution File (Assignment7.py)

Controls driver lifecycle, instantiates page objects, performs business actions, and validates test assertions.

```

PS C:\Users\roshn\OneDrive\Documents\Desktop\ALL SELENIUM\seleniumassignment\Assignment 2nd wk> python Assignment7.py
Login successful
Dashboard title: Products

```

Screenshot 4: Terminal Execution Result

Console execution output displaying successful login assertion and dashboard header title verification.

6. BRIEF RESULT, OBSERVATION, AND CONCLUSION

Result: The script executed cleanly with zero assertion errors, successfully printing standard login execution messages and reading Dashboard title: Products from the live web application.

Observation: Utilizing tuple unpacking (`*self.locator`) allowed seamless argument passing into Selenium's `find_element()` method. By decoupling assertions from locator declarations, page objects remain purely transactional and reusable across multiple test scenarios.

Conclusion: The Page Object Model architectural pattern was successfully implemented. The resulting framework exhibits high maintainability, clean separation of concerns, and robust execution standards suitable for enterprise-level automation suites.